

RK7 Implementation and CPMG Analysis

Donghun Jung

20 Nov 2025

Department of Physics, Sungkyunkwan University
Center for Quantum Technology QuiME Lab, Korea
Institute of Science Technology



TODO

1. **Analytical study:** Find better parameters

2. **Polish code:**

- Memory management
- Find better hyperparameters for faster simulation
- Reduce simulation time to ~ 1 min (for learning purposes)

3. **Run learning** after obtaining good parameters

4. **(Later) Develop density matrix time-evo solver in PyTorch**

- NV/nuclear spins are not fully initialized to pure states
- Need to consider mixed states $\rightarrow$ density matrix approach

Progress Update

Completed Tasks

- **Code optimization**
 - Memory management improvements
 - Hyperparameter tuning for faster simulation
 - Reduced simulation time to ~ 1 min (for learning purposes)
- **Density matrix/Unitary matrix solver development**
 - Implemented time-evolution solver in PyTorch
 - Supports mixed states for non-initialized NV/nuclear spins

In Progress

- **Numerical/Analytical analysis of CPMG**

Progress Update

Pending(DDrf)

- **Analytical study:** Determine better control parameters
- **Run learning:** After obtaining optimized parameters

Outline

- Progress Update
- Solving Differential Equations
 - Euler Method & Runge-Kutta Method
 - Key Features of RK7 Implementation
 - Why PyTorch Implementation Was Necessary
- CPMG
 - CPMG Sequence
 - Finding τ Analytically
 - Conditional Operation
- CPMG Analysis
 - Effect of Non-negligible MW Pulse Time
 - Rotation Axis Analysis ($\vec{\sigma}_0 \cdot \vec{\sigma}_1$)
 - Singular Value Analysis (λ_i)
- Future Work

Solving Differential Equations

Time-Dependent Schrödinger Equation

$$\frac{\partial}{\partial t}|\psi\rangle = -i\mathcal{H}(t)|\psi\rangle$$

Naive Approach (Euler Method)

Consider a very simple model:

$$\mathcal{H} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

and initial state $|\psi\rangle = |0\rangle$. We can do something like this:

$$|\psi(t + dt)\rangle = |\psi(t)\rangle - idt\mathcal{H}|\psi(t)\rangle$$

(General idea of the Euler method)

(Explicit) Runge-Kutta Method

The generalization of explicit Runge--Kutta methods is given by

$$y_{n+1} = y_n + dt \sum_{i=1}^s b_i k_i,$$

where

$$k_1 = f(t_n, y_n),$$

$$k_2 = f(t_n + c_2 dt, y_n + (a_{21} k_1) dt),$$

$$k_3 = f(t_n + c_3 dt, y_n + (a_{31} k_1 + a_{32} k_2) dt),$$

$$\vdots$$

$$k_s = f(t_n + c_s dt, y_n + (a_{s1} k_1 + a_{s2} k_2 + \cdots + a_{s,s-1} k_{s-1}) dt).$$

I ported code from `qutip/solver/integrator/qutip_integrator.py`. The specific coefficient values are shown [here](#).

Key Features of Implementation and Improvement

Feature	Description
Modified Norm	Error-controlled stepping with WRMSE norm
Memory Pooling	Pre-allocated buffers minimize additional GPU memory allocations
GPU Acceleration	Full PyTorch tensor operations with CUDA support
Fixed Time Step	Removed adaptive time stepping; <code>dt</code> is not re-estimated at every step
Event-Adaptive Stepping	The solver automatically reduces step size during critical intervals

Key Features of Implementation and Improvement

Modified Norm

The solver uses weighted root mean square error norm:

$$\text{error} = \sqrt{\frac{1}{N} \sum_i \left(\frac{|\Delta y_i|}{\text{atol} + \text{rtol} \cdot |y_i|} \right)^2}$$

This norm does not explicitly test whether the state vector norm is preserved at 1. Nevertheless, the norm remains preserved if `dt` and total evolution time are chosen properly. This also allows us to extend the RK7 implementation beyond SESolver to other solvers.

Fixed Time Step

With fixed time stepping, we must verify that the final state remains unchanged when `dt` is reduced (i.e., the solution has converged). However, this approach reduces execution time.

Key Features of Implementation and Improvement

Module Structure is shown below:

```
solver/
├── __init__.py      # Package exports
├── solver.py        # Base Integrator class
├── explicit_RK.py   # Core RK algorithm
├── rk7_coeff.py     # Verner 7 coefficients
├── sesolver.py      # Schrödinger equation solver
├── lvnesolver.py    # Lindblad von Neumann equation solver
└── upsolver.py      # Unitary propagator solver
```

Key Features of Implementation and Improvement

SESolver - Schrödinger Equation: Solves the time-dependent Schrödinger equation:

$$i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = H(t) |\psi(t)\rangle$$

LVNESolver - Lindblad von Neumann Equation: Solves the master equation for density matrices:

$$\frac{\partial \rho}{\partial t} = -\frac{i}{\hbar} [H, \rho] + \mathcal{L}[\rho]$$

UPSolver - Unitary Propagator: Evolves the unitary propagator matrix:

$$i\hbar \frac{\partial U}{\partial t} = H(t) U(t)$$

Why It Was Necessary

- **PyTorch enables GPU integration**
 - We plan to realize GHZ states for up to (1+8)-qubit systems.
 - QuTiP's C-implemented solver would face scaling issues at this size.
- **PyTorch enables optimization with convenient and efficient optimizers**
 - DDrf requires tuning: DDrf is too complex to selectively control each nuclear spin qubit for precise phase updates.
 - Pulse optimization: Implementing two-qutrit QST POVM operators in $2 \mu\text{s}$ requires smoother pulses, which can be derived from the GRAPE algorithm.

Note: QuTiP has supported GPU since May 2024 (v5.0.2, via TensorFlow)

Dynamic Decoupling

Original Purpose: Extending Coherence Time

CPMG was originally used to increase coherence time, but can also be used for various purposes, including implementing conditional gates.

Hamiltonian

$$\mathcal{H} = \underbrace{\gamma_c B_z I_z^i + A_{||}^i S_z I_z^i + A_{\perp} S_z I_x^i}_{\text{Drift Hamiltonian}} + \underbrace{\frac{1}{\sqrt{2}} \Omega_{\text{MW}} S_x}_{\pi\text{-pulse}}$$

$$\rightarrow \mathcal{H} = |0\rangle\langle 0| \otimes \omega_0 I_z + |-1\rangle\langle -1| \otimes (\omega_L I_z - A_{||} I_z - A_{\perp} I_x)$$

$$\rightarrow \mathcal{H} = |0\rangle\langle 0| \otimes H_0 + |-1\rangle\langle -1| \otimes H_1$$

That is, nuclear spin evolves by $e^{-iH_{0(1)}t}$ when electron spin is in state $|0\rangle(|1\rangle)$.

Figure. Diagram for CPMG sequence/ CPMG is a repetition of $(\tau - \pi - 2\tau - \pi - \tau)$ for $\frac{N}{2}$ times. The figure is extracted from *Phys. Rev. X* 15, 021011.

CPMG Sequence

- Initial electron spin state 0(1): $e^{-i\phi\hat{I}\cdot\hat{\sigma}^i}$
 - if NV spin is 0-state: $e^{-i\phi\hat{I}\cdot\hat{\sigma}^0} = e^{-iH_0\tau}e^{-iH_12\tau}e^{-iH_0\tau}$
 - if NV spin is 1-state: $e^{-i\phi\hat{I}\cdot\hat{\sigma}^1} = e^{-iH_1\tau}e^{-iH_02\tau}e^{-iH_1\tau}$
- When vectors σ^0 and σ^1 are anti-parallel $\rightarrow$ conditional gate
- Directions are generally parallel, but become anti-parallel at specific τ values

Note: ϕ is the same regardless of whether the initial state is 0 or 1 in the CPMG train.

Finding τ Analytically(?)

To enable a conditional gate via repetition of CPMG train, we should choose τ such that $\hat{\sigma}^0 \cdot \hat{\sigma}^1 = -1$. τ can be found analytically under a strong magnetic field. [\[Phys. Rev. Lett. 109, 137602\]](#)

$$\tau \simeq \frac{(2k-1)\pi}{2\gamma_c B_z + A_{||}}$$

Then, the number of repetitions of CPMG train is chosen meticulously to achieve $N\phi = \frac{\pi}{2}$.

Conditional Operation

At a certain τ , we observe a conditional operation. Red (Blue) arrow denotes the rotation axis when the initial electron spin state was $|0\rangle$ ($|1\rangle$).

CPMG analysis

- Initial electron spin state 0(1): $e^{-i\phi\hat{I}\cdot\hat{\sigma}^i}$
 - if NV spin is 0-state: $U_0 = e^{-i\phi\hat{I}\cdot\hat{\sigma}^0} = e^{-iH_0\tau}e^{-iH_12\tau}e^{-iH_0\tau}$
 - if NV spin is 1-state: $U_1 = e^{-i\phi\hat{I}\cdot\hat{\sigma}^1} = e^{-iH_1\tau}e^{-iH_02\tau}e^{-iH_1\tau}$

If one CPMG train can be described this way, the unitary gate takes the form of a block matrix:

$$U^{\otimes \frac{N}{2}} = \begin{pmatrix} U_0^{\otimes \frac{N}{2}} & 0 \\ 0 & U_1^{\otimes \frac{N}{2}} \end{pmatrix}$$

where off-diagonal components are zero.

However, if the time for MW implementing π -pulse gate is not short enough, it can cause additional effects beyond those described below.

CPMG analysis

However, if we calculate U with non-negligible MW time, off-diagonal components become non-zero and each block matrix is no longer unitary.

$$U^{\otimes \frac{N}{2}} = \left(e^{-i\tau H_0} e^{-i\tau_{\text{MW}}(H_0 + H_{\text{MW}})} e^{-i2\tau H_0} e^{-i\tau_{\text{MW}}(H_0 + H_{\text{MW}})} e^{-i\tau H_0} \right)^{\otimes \frac{N}{2}} = \begin{pmatrix} V_{00} & V_{01} \\ V_{10} & V_{11} \end{pmatrix}$$

Its operation on a state is described as:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |\phi\rangle$$

then, we have

$$|\psi\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle \otimes \underbrace{(V_{00} + V_{01})}_{=V^0}|\phi\rangle + |1\rangle \otimes \underbrace{(V_{11} + V_{10})}_{=V^1}|\phi\rangle)$$

CPMG analysis

Both V_0 and V_1 are not unitary. To analyze them, I used polar decomposition which is related to SVD.

$$V = UP = (uv^\dagger)(v\Sigma v^\dagger)$$

where u, v are unitary matrices derived from SVD. Here U is unitary, while P is non-unitary and can be regarded as a quantum channel.

$$P = \sum_i \lambda_i |\lambda_i\rangle \langle \lambda_i|$$

where λ_i are the singular values. Thus, we have

$$|\psi\rangle \rightarrow \frac{1}{\sqrt{2}} (|0\rangle \otimes U_0 P_0 |\phi\rangle + |1\rangle \otimes U_1 P_1 |\phi\rangle)$$

CPMG analysis

We can analyze from two perspectives:

- How $\vec{\sigma}_0 \cdot \vec{\sigma}_1$ (with respect to U_0 and U_1) changes with t_{MW} and N .
- How the singular values $\lambda_{0,1}$ change with t_{MW} and N .

For this tentative simulation, I used the following parameters:

Param	Value	Description	Param	Value	Description
B_z	440.1 G	Magnetic field strength	τ	0.1 - 20 μs	Free evolution time (varied)
$A_{\parallel}$	(2π) 130 kHz	Parallel hyperfine coupling	τ_{MW}	0.1 - 50 ns	π -pulse duration (varied)
$A_{\perp}$	(2π) 50 kHz	Perpendicular hyperfine coupling	N	2 - 40	Number of CPMG repetitions

CPMG analysis: $\vec{\sigma}_0 \cdot \vec{\sigma}_1$

The following colormap shows how the $\vec{\sigma}_0 \cdot \vec{\sigma}_1$ value changes with τ and τ_{MW} .

τ shifts to shorter values as τ_{MW} increases. However, the amount of left-shift keeps the total time for CPMG train constant.

CPMG analysis: $\vec{\sigma}_0 \cdot \vec{\sigma}_1$

Varying N and fixing τ_{MW} , the value τ that makes total operation conditional, $\vec{\sigma}_0 \cdot \vec{\sigma}_1 = -1$, is unchanged.

CPMG analysis: λ_i

If $\lambda_i \simeq 1$, P can be approximated as the identity matrix.

$$P = \sum_i \lambda_i |\lambda_i\rangle \langle \lambda_i|$$

This operator amplifies or attenuates components based on λ_i . While the overall operation preserves norm, P alone does not preserve the norm of the state vector.

Quick example:

Suppose $\lambda_i = \left\{ \sqrt{\frac{3}{2}}, \sqrt{\frac{1}{2}} \right\}$, with $|\lambda_0\rangle = |0\rangle$ and $|\lambda_1\rangle = |1\rangle$.

If the prepared state is $|+\rangle$, it transforms to

$$|+\rangle \rightarrow \frac{\sqrt{3}}{2} |0\rangle + \frac{1}{2} |1\rangle$$

CPMG analysis: λ_i

As expected, as τ_{MW} and N increase, the singular value λ_i deviates from 1.

[TODO] CPMG analysis: λ_i

In a CPMG sequence, we perform an X -basis measurement at the end of the sequence. Without considering the drift Hamiltonian during π -pulse time, we have

$$\begin{aligned} P_x &= \text{Tr}(|+\rangle\langle+| \otimes I) \rho = \text{Tr}(|+\rangle\langle+| \otimes I) U \rho_0 U^\dagger \\ &= \frac{1}{4} \text{Tr} U_0 \rho_{^{13}\text{C}} U_0^\dagger + \frac{1}{4} \text{Tr} U_0 \rho_{^{13}\text{C}} U_1^\dagger + \frac{1}{4} \text{Tr} U_1 \rho_{^{13}\text{C}} U_0^\dagger + \frac{1}{4} \text{Tr} U_1 \rho_{^{13}\text{C}} U_1^\dagger \\ &= \frac{1}{2} \left(\Re(\text{Tr} U_0 U_1^\dagger) + 1 \right) \end{aligned}$$

where

$$\rho_0 = |+\rangle\langle+| \otimes \rho_{^{13}\text{C}}$$

and the last line follows from $\rho_{^{13}\text{C}} = I$.

[TODO] CPMG analysis: λ_i

Now it becomes more complex...

$$\begin{aligned} P_x &= \text{Tr}(|+\rangle\langle+| \otimes I) \rho = \text{Tr}(|+\rangle\langle+| \otimes I) U \rho_0 U^\dagger \\ &= \text{Tr} \frac{1}{4} \begin{pmatrix} I & I \\ I & I \end{pmatrix} \begin{pmatrix} V_{00} & V_{01} \\ V_{10} & V_{11} \end{pmatrix} \begin{pmatrix} \rho & \rho \\ \rho & \rho \end{pmatrix} \begin{pmatrix} V_{00}^\dagger & V_{10}^\dagger \\ V_{01}^\dagger & V_{11}^\dagger \end{pmatrix} \\ &= \frac{1}{4} \text{Tr}(\sigma_{00} + \sigma_{01} + \sigma_{10} + \sigma_{11}) \end{aligned}$$

where

$$\begin{aligned} \sigma_{00} &= (V_{00} + V_{01}) \rho^{13}\text{C} (V_{00}^\dagger + V_{01}^\dagger) &= U_0 P_0 \rho^{13}\text{C} P_0 U_0^\dagger \\ \sigma_{01} &= (V_{00} + V_{01}) \rho^{13}\text{C} (V_{10}^\dagger + V_{11}^\dagger) &= U_0 P_0 \rho^{13}\text{C} P_1 U_1^\dagger \\ \sigma_{10} &= (V_{10} + V_{11}) \rho^{13}\text{C} (V_{00}^\dagger + V_{01}^\dagger) &= U_1 P_1 \rho^{13}\text{C} P_0 U_0^\dagger \\ \sigma_{11} &= (V_{10} + V_{11}) \rho^{13}\text{C} (V_{10}^\dagger + V_{11}^\dagger) &= U_1 P_1 \rho^{13}\text{C} P_1 U_1^\dagger \end{aligned}$$

Future Work

- Numerical/Analytical Study
 - Derive analytical expression for P_x with non-negligible MW time
- DDrf Study for GHZ study
- Further Entanglement study